# Solutions & Portfolios

## portfolio

## User Manual

Aggregated flow reports across multiple ARTs, solutions and portfolios

---

situation-report — github.com/Jaegerfeld/situation-report

For Agile Coaches, RTEs and PI Managers — Version 0.43.0

# 1. What is 'Solutions & Portfolios'?

Until now SituationReport always looked at one team group or ART. The Solutions & Portfolios module (technically: `portfolio`) combines several already-configured ARTs into one shared report — at Large-Solution or portfolio level.

You configure each ART once as usual (in build_reports, as a project template). After that you only state which ARTs belong to a solution — and get an aggregated report across all of them.

> In short: a solution = several ARTs combined. A portfolio = several solutions combined. The ART reports themselves do not change; they are only referenced.

# 2. Key concepts

| Term | Meaning |
|---|---|
| ART | Agile Release Train / team group — the level you already analyse today in build_reports. |
| Solution | A grouping of several ARTs. Refers to their project templates. |
| Portfolio | A grouping of several solutions (nested: portfolio > solutions > ARTs). |
| Template | A saved configuration. ARTs are included via their build_reports template; a solution can itself be saved as a template and referenced by a portfolio. |
| Pooled | Mode: all issues are merged into one dataset — the solution seen as a single system. |
| Comparison | Mode: each unit (ART or solution) is shown separately, side by side. |

# 3. Starting

## 3.1 From the launcher

Start SituationReport. The entry screen is split: left 'Large Solutions & Portfolios', right the familiar ART tools. Click the Solutions & Portfolios card on the left.

## 3.2 Directly

Alternatively, without the launcher:

```
python -m portfolio
```

*Without arguments the graphical interface opens. With arguments the command-line interface runs (see section 6).*

# 4. The interface step by step

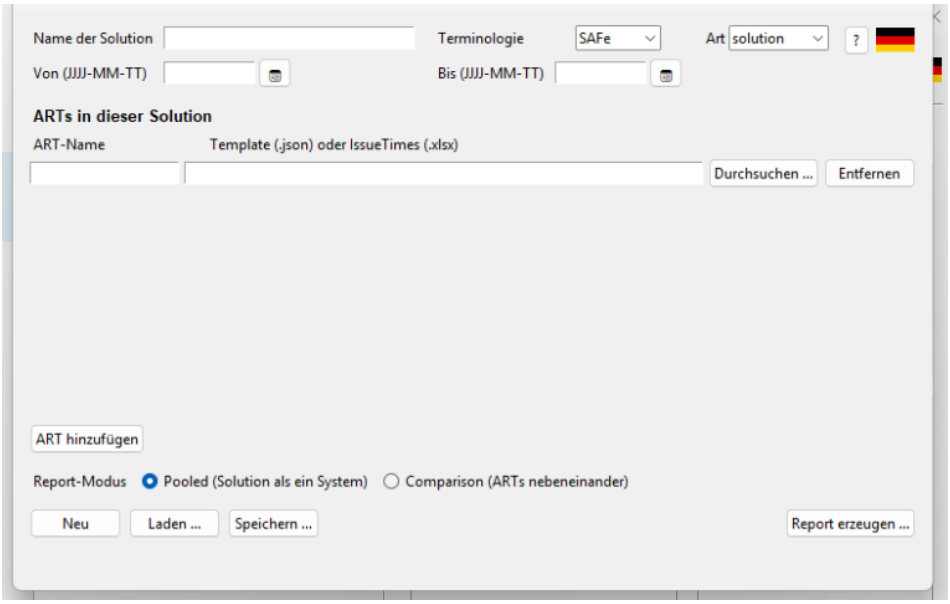



*Fig. 1: The Solutions & Portfolios interface.*

| Field / action | Description |
|---|---|
| Name | Name of the solution or portfolio (shown in the report). |
| Terminology | SAFe or Global — labelling of the report metrics. |
| Kind | solution (members are ARTs) or portfolio (members are solutions). |
| From / To | Optional reporting period (YYYY-MM-DD). |
| Add ART | One row per member: name + source. For 'solution' the source is an ART template (.json) or an IssueTimes.xlsx directly; for 'portfolio' a solution template (.json). |
| Browse | Pick the source file. |
| Report mode | Pooled or Comparison (see section 5). |
| Save | Save the configuration as JSON (= a reusable template). |
| Load | Open a saved configuration. |
| Data sources … | Dialog with the solution's ten register paths (risks, NFR, capabilities, dependencies, decisions, SLO, DORA, flow problems, themes, harmonisation folder) plus a per-field status (found/empty/MISSING). 'Take from data folder …' fills the fields by convention from registers/; for a portfolio the dialog only shows which registers the member solutions bring. On save, paths inside the config folder are relativised automatically (Datenraum). |
| Generate report | Build the report. Extension .html = web page, .pdf = PDF. It opens afterwards. |

Tip: a saved solution file is your solution template. To build a portfolio, choose Kind = portfolio and point each member at such a saved solution file.

Portfolio data folder: when config and data live in one shared folder, every path in the config may be relative — it resolves against the config file's folder; absolute paths stay unchanged. The folder then moves, copies and zips as a whole. The test-data generator's demo scenario produces exactly this structure: solutions/<name>/ with solution.json, arts/ and registers/ (standard names), plus snapshots/ and raw/.

# 5. The report

## 5.1 Management summary

At the very top sits a compact figures table — one row per unit (in pooled mode a single row for the whole solution/portfolio):

| Figure | Meaning |
|---|---|
| Items | Number of issues in the period. |
| Completed | Of those, the closed ones. |
| Open (WIP) | Open issues (still in progress). |
| Median / 85th / 95th CT | Cycle time in days — median plus 85th and 95th percentile. |
| <= Nd | Share of completed issues within the target (Target CT, default 90 days). |
| Median / 85th LT | End-to-end lead time (Created to Closed) in days — in pooled mode the solution lead time across all ARTs. |

## 5.2 Pooled vs. comparison

Pooled answers: How is the solution doing as a whole? All issues are merged and analysed together.

Comparison answers: Which unit is the outlier? For a solution each ART is shown separately, for a portfolio each solution. Charts are placed side by side per metric.

ART depth (optional; checkbox „Evaluate down to ART level“ or `--art-depth`) answers: Which ART is the outlier? A portfolio then compares the individual ARTs instead of its member solutions — labelled „Solution · ART“ so that equally named ARTs of two solutions stay distinguishable. Only this makes the two workflow-bound „ART & Teams“ analyses (Process Flow) possible at portfolio level at all: they need the individual ART's transitions, which pooling discards. In pooled mode the charts stay pooled — there ART depth adds its own „ART Detail“ table per ART.

## 5.3 The metrics

| Metric | What it shows |
|---|---|
| Flow Velocity | Throughput: completed items per period/PI. |
| Flow Time | Cycle-time distribution (boxplot, scatter with trend). |
| Flow Distribution | Split by issue type and stage shares. |
| CFD | Cumulative Flow Diagram (see 5.4). |
| Flow Load | Aging WIP — comparison mode per ART only, as it is workflow-dependent. |

## 5.4 CFD across different workflows

Different ARTs often use different workflow stages. To make a shared CFD possible, the module automatically maps every stage to one of three canonical groups — To Do, In Progress, Done — and sums the counts per group. This keeps the CFD comparable across ARTs with different workflows.

## 5.5 Data quality & confidence

Below the management summary, the Data Quality per Source table shows one row per data source: record count, its share of all items, the share without a First Date, the open share, whether CFD data was supplied, the data freshness — and a traffic-light confidence:

| Level | Meaning |
|---|---|
| high | Source complete and fresh — figures trustworthy. |
| medium | More than 10 % without a First Date, no CFD data, or data older than 30 days. |
| low | No records, or more than half without a First Date — cycle-time statements would rest on a minority of the data. |

The table title carries the coverage ratio ('x/y sources delivered data'). In the PDF the table is page 2. In comparison mode, outliers are additionally marked: Median-CT and 95th-percentile cells turn red when they exceed 1.5x the column median (three units minimum).

## 5.6 Custom stage map (optional)

Instead of the fixed three groups, the solution configuration may define its own canonical stages — an optional `stage_map` block with an ordered source-stage assignment and the `first_stage`/`closed_stage` markers for the CFD boundaries. Unmapped stages fall into first_stage with a warning; configurations without the block behave unchanged.

## 5.7 ROAM risk board (optional)

When the solution configuration references a risk register via the `risks` field (JSON with id, title, roam, owner, impact, status_since), the report renders a ROAM board below the quality table: rows in R-O-A-M order (Resolved / Owned / Accepted / Mitigated) with coloured category and impact cells. An owned risk sitting in its category for more than 30 days gets a red Since cell (aging) — ownership

without movement becomes visible. A portfolio aggregates the registers of all member solutions and adds a Solution column. Owners are teams, not persons. A missing or invalid file is logged and skipped; in the PDF the board is its own page.

## 5.8 NFR/runway dashboard (optional)

When the solution configuration references an NFR register via the `nfr` field (JSON with nfrs and runway blocks), the report renders two tables below the ROAM board: the NFRs (target/actual/status met, at_risk, violated) and the architecture-runway elements (status in_place, building, gap with a needed_by date). People assess the statuses in PI planning/review — the tool does not compute target vs. actual itself. Violated NFRs and gaps sort first; a runway element whose needed_by has passed while not in place renders as overdue (red date cell). A portfolio aggregates the registers of all member solutions with a Solution column. Owners are teams, not persons.

## 5.9 Capability map & health (optional)

When the solution configuration references a capability map via the `capabilities` field (JSON with id, title, health, arts, owner, assessed_on), the report renders a table of the business capabilities: a health traffic light healthy/at_risk/critical (critical first, assessed by people in PI planning/review) and the contributing ARTs. A capability with no contributing ART is flagged as uncovered — business value nobody delivers; unknown ART names produce a drift warning in the log. A portfolio aggregates the maps of all member solutions with a Solution column. The capability map (business capabilities) is not the stage_map (workflow stages) — different dimension, different source. Owners are teams, not persons.

## 5.10 Dependency heatmap (optional)

When the solution configuration references a dependency register via the `dependencies` field (JSON with id, title, from, to, status blocked/at_risk/on_track/done, due), the report renders a heatmap plus a detail table: the heatmap counts open dependencies per from/to pair, each cell carrying the colour of its most urgent status; in the table, blocked ones sort first, and a dependency whose due date has passed while not done renders as overdue (red date cell). The target (to) may lie outside the solution — another solution's ART, a vendor, an external system; cross-solution dependencies become visible in the portfolio report. Cross-ART dependencies are system behaviour — making them visible protects against local optimisation.

## 5.10a Decision point (cross-value-stream pressure)

Right below the heatmap sits the decision point: a measure of how much the seams BETWEEN the portfolio's value streams are under pressure. It answers the question the calendar usually answers in the ritual — do we need to convene a Value Stream Conference?

It is computed over all open dependencies whose target belongs to another solution of the same portfolio: status weight (blocked 3, at_risk 2, on_track 1) × overdue factor (on time 1, overdue 2, more than 30 days overdue 3). Whole numbers, so a human can recompute the value by hand. Which ART

belongs to which solution is derived from the configuration, never asserted in the register.

The threshold is agreed by the staff, not by the tool: field `report.cross_vs_threshold`, the GUI field “VSC threshold”, or `--cross-vs-threshold N`. Without an agreed threshold the pressure is reported but never alarms — and an unusable entry (0, negative, text) counts as “not agreed” rather than as an alarm nobody decided on.

Three rules keep the alarm quiet. First, only portfolio-internal seams wake anyone: a dependency on a vendor or external system is real pressure, but no conference of these value streams can decide about it — it is reported separately and excluded from the indicator. Second, the tool invents no threshold. Third, the below-threshold case is stated out loud rather than omitted: an indicator that appears only when things are bad teaches its readers to read absence as “not computed”.

The block appears in the report and the PDF, not in the conference pre-read: the pre-read is the material for a conference already convened — a wake-up call belongs where you look before that. A single solution has no seam between value streams; there the block says so explicitly instead of showing a reassuring zero.

## 5.11 Decision / assumption log (optional)

When the solution configuration references a decision log via the `decisions` field (JSON with id, kind decision/assumption, title, status, owner, logged_on, review_by, supersedes), the report renders a lightweight ADR-style log table. Decisions carry the statuses proposed/accepted/superseded, assumptions open/confirmed/invalidated — the parser enforces the matching set, and supersedes must name an entry of the same log (the trade-off trail stays intact). An open assumption whose review_by date has passed sorts first and gets a red 'review due' cell — the hook for red-team and premortem sessions. A portfolio aggregates the logs of all member solutions with a Solution column. Owners are teams, not persons.

## 5.12 Delta briefing (comparing snapshots)

`--snapshot FILE` freezes the computed report state (metrics per unit and pooled, source confidence, all seven governance registers incl. flow problems and SLOs) into a small JSON; `--as-of` pins the observation date. `--delta PREV NOW` compares two snapshots without a config and answers 'what changed?': metric deltas at display precision, throughput in the period, confidence transitions per source, per register the added/removed entries and status transitions (worsenings first, red) plus newly overdue items — judged against each snapshot's as-of date, so only genuine flips count. Output: --output *.md writes Markdown, any other suffix the HTML page, no --output prints to stdout. A delta with no changes says so explicitly. All of it deterministic — the optional AI narration (section 5.16) may rephrase, never produce numbers. In the GUI: 'Save snapshot …' freezes the currently configured solution, 'Delta briefing …' asks for the earlier and later snapshot and opens the briefing in the browser.

## 5.13 SLO & DORA registers from external sources (C1/C2)

Via the config fields `slo` and `dora` the report shows two more views: 'Service Levels & Error Budgets' (per SLO the target, SLI, remaining error budget and status met/at risk/breached — breached first; the

budget rule is central and identical for every source) and 'Delivery Performance (DORA) & Code Quality' (the four DORA keys per unit, tiered at the published thresholds, overall tier = worst metric; coverage, maintainability and critical issues below). The registers are produced by the pluggable sources framework: `python -m sources fetch --kind slo --config sources.json --output slo.json` — providers are exchangeable and combinable (several sources fill ONE register, each row keeps its origin): file (universal, no API approval needed), prometheus, github, gitlab, sonarqube. A new source is a single file in sources/providers/ (recipe on the module page). Tokens only via environment variables, never stored.

## 5.14 Flow-problem backlog & conference pre-read (B6, VSC)

Via the config field `flow_problems` the report shows the Value-Stream Conference's impediment backlog: per problem the status (open/committed/resolved/dropped), affected value streams (cross-VS is derived: more than one stream), who raised it, the owning team, commitment and follow-up PI — plus a conference counter. The workshop pattern 'logged, never mitigated, back next PI' becomes measurable: unresolved problems from the third conference on sort first with a red counter. The conference pre-read (`--conference preread.html --conference-date YYYY-MM-DD`) bundles the meeting inputs printably: current data, impediment backlog with ROAM/dependencies, business objectives (capabilities + SLOs) and the integrated roadmap (input 4, B7); since 0.36.0 the harmonisation matrix as an annex when the solution names a workspace (field harmonisation). With --narrate five AI drafts (introduction and one paragraph per input) go to preread.html.narration.md; --conference-draft inline also places them into the page (D8, 0.37.0).

The conference date (0.31.0): the pre-read is written for a planned date, not for the day it was produced. The Conference on field in the manager window stores it in the configuration; `--conference-date` overrides it for a single run. The header names the date with its remaining lead time, and the file name follows it. Without a date the header says so rather than showing today's date, which would claim a conference held today.

The colour legend (0.31.0): every shaded cell draws from the same five backgrounds. Each page therefore opens with a colour key (green on plan, blue committed, yellow watch, red critical, grey closed), and every shaded table carries a line with its own status words underneath — needed because the same colour means different things per register: grey is 'done' for a dependency and 'accepted' for a ROAM risk. The same holds for the full report and the delta briefing.

The report language (0.32.0): report labels and texts follow the language of the window — set the interface to French and the report comes out in French. To pin a report to one language, pick it in the Report language field; the default '—' means 'follow the interface' and stores nothing. Precedence: `--lang` beats `report.language` beats the window language; without any of the three, English. The date format follows — up to 0.31.0 every report carried the German format, including the English one. Not translated: the metric names (still governed by the SAFe/Global switch), the axis labels inside the embedded charts, and the Markdown form of the delta briefing.

## 5.15 Strategic themes & integrated roadmap (B7, VSC)

Via the config field `themes`, strategic themes get a structured home and the solution level its integrated roadmap: themes with epic links; epics with train, horizon (near-term granular, far-term coarse: P1 · P2 · Y1 · Y2 · Y3) and status. Orphan detection in both directions: a theme no epic pays into is 'declared & forgotten' (red; judged portfolio-wide), an epic with an empty theme field is a zombie initiative (red in matrix and zombie list) — a typo in the reference is a validation error instead. The roadmap matrix shows trains × horizons; the conference pre-read carries it all as input 4. Roadmap epics flow into D2 snapshots: the delta briefing documents the 'updated roadmaps' per conference (horizon shifts, status changes; a lost theme reads as '→ zombie').

## 5.16 AI narration of the delta briefing (D2, optional)

`--narrate [PROVIDER]` adds the section 'Narration (Entwurf)' to the delta briefing: 5–8 sentences on 'what changed, what deserves attention?', phrased by a language model — locally via ollama by default (model mistral-nemo; data never leaves the machine), alternatively claude (Anthropic API; key ONLY from the environment variable ANTHROPIC_API_KEY) or mock (a stand-in for demos and tests, no model at all); `--llm-model`, `--llm-base-url` and `--llm-lang de|en` refine. Three things are wired in and cannot be bypassed: the numbers guard discards any text containing numbers that do not occur in the briefing ('the LLM writes, it does not calculate'); the AI label (Art. 50 EU AI Act) visibly marks every draft as unreviewed — only the editing human approves; the operator evidence (llm_audit.jsonl next to the output) logs every request with model, deployment class, prompt version and SHA-256 hashes — never full texts, never keys. Markdown output additionally writes <output>.narration.md as the editable draft. GUI: checkbox 'AI narration (draft)' plus a provider picker next to the delta-briefing button. With more than one model in Ollama, the Model field beside it picks one; it offers the installed models but stays typable, and empty means: let the provider decide. If Ollama runs on another machine, the Address field takes it (otherwise the OLLAMA_HOST environment variable applies, else http://localhost:11434). Provider, model and address belong to this machine: they are remembered for the next start but deliberately do NOT go into the saved solution configuration you hand on. `python -m llm models` lists what is installed. Exporting the prompt (0.42.0): provider 'export' calls no model; it writes system prompt and contract to a file next to the output and to the clipboard, to be run on another machine or model; the answer goes into the draft file. Conference pre-read (D8, 0.37.0): with the checkbox ticked, five labeled drafts — an introduction plus one paragraph per input — go to the file preread.html.narration.md; the box 'Pre-read draft' optionally places them into the pre-read as well. The file is the default, because the pre-read is the paper of an already convened conference and is read aloud. Where the checkbox does NOT apply: PDF output, which has nowhere to put the editable draft — the status line says so instead of ignoring the checkbox silently. Setting up Ollama: separate guide ollama_Installationsanleitung_EN.pdf, or docs site → Tutorials. Without --narrate the briefing stays fully deterministic. Since D1, --narrate also acts on the REPORT run (HTML output): right below the Management-Summary table the labeled section 'Executive Summary (Entwurf)' appears — its input is the deterministic metrics contract (metrics, source confidence, governance head counts; structurally without persons), plus <output>.exec_summary.md for editing. In the GUI the same checkbox also applies to 'Generate report …'; if the AI fails, the report is written without the draft (clean degradation). Multilingual delivery (D6): --translate LANG … additionally delivers the primary text in the house languages de/en/ro/pt/fr — the draft when narrated, otherwise the deterministic delta briefing itself; every translation is guarded (numbers invariant), labeled in the TARGET language and audited. Edited,

approved texts are translated in one step via python -m llm translate FILE --to …. Red-team questions (D5): --red-team questions.md generates premortem and attack questions from the decision/assumption log — ONLY questions, never recommendations (a questions guard machine-discards anything else); raw material for humanly moderated sessions, labeled and audited.

### 5.17 Situation-briefing one-pager (P6)

One slide everyone can read at once. `--onepager page.html` renders four dimensions (flow, quality, value, dependencies) x three questions (where do we stand, what has changed, what follows) on one A4 page, BLUF on top, confidence per cell. Column 2 fills only with `--since-snapshot`; column 3 names decision points only (wake-up call, surviving flow problems, violations, overdue items) and leaves the presenter a line - no recommendation. `--kata` (GUI box Kata) renders the empty practice sheet of the curriculum. GUI: button One-pager ...

## 6. Command line (CLI)

For automation and reproducible reports:

```
python -m portfolio my_solution.json --output report.html
python -m portfolio my_solution.json --mode comparison --pdf report.pdf
```

| Option | Meaning |
|---|---|
| <config> | Path to the solution/portfolio configuration (JSON). |
| --output FILE | Write the HTML report to this file. |
| --pdf FILE | Write a multi-page PDF report. |
| --mode pooled\|comparison | Aggregation mode (default: pooled). |
| --art-depth | Evaluate down to ART level (default: off). |
| --cross-vs-threshold N | Alarm threshold of the decision point. |
| --metrics ID ... | Specific metrics (default depends on the mode). |
| --terminology SAFe\|Global | Labelling mode (default: SAFe). |
| --target-ct DAYS | Target cycle time in days (default: 90). |
| --browser | Open the generated HTML report in the browser. |

## 7. Frequently asked questions (FAQ)

### Do I have to reconfigure the ARTs?

No. The module references the existing ART templates. Each ART template remains the single source of truth.

### Why is 'Flow Load' missing from the pooled report?

Flow Load groups by current stage. Across different workflows that is not comparable, so it appears only in comparison mode per ART.

### Why is the target-CT share low or shown as a dash?

A dash means there are no completed issues with a valid cycle time in the period. A low percentage means many issues take longer than the target.

### How do I create a portfolio?

First save each solution. Then create a new configuration with Kind = portfolio and point each member at a saved solution file.

## 8. Glossary

| Term | Explanation |
|---|---|
| ART | Agile Release Train — multi-team structure in SAFe. |
| CFD | Cumulative Flow Diagram — cumulative issue count per stage. |
| Cycle Time | Time from the first active stage to completion. |
| PI | Program Increment — fixed planning period (often 8 to 12 weeks). |
| Pooling | Merging several datasets at the issue level. |
| WIP | Work in Progress — open, not-yet-completed issues. |